

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

Lopamudra Patro

(1WA24CS158)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Lopamudra Patro (1WA24CS158), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Ramya K M
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	(a)Write program to obtain the Topological ordering of vertices in a given digraph. (b)Leetcode Problem 207- Course schedule	1 - 4
2	Implement Johnson Trotter algorithm to generate permutations.	5 - 6
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	7 - 9
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	10 - 12
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	13 - 15
6	(a)Implement 0/1 Knapsack problem using dynamic programming. (b)Leetcode Problem 70 - Climbing Stairs	16 - 18
7	(a)Implement All Pair Shortest paths problem using Floyd's algorithm. (b)Leetcode Problem 1334 - Find the City With the Smallest Number of Neighbors at a Threshold Distance	19 - 22
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	23 - 26
9	Implement Fractional Knapsack using Greedy technique.	27 - 28
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	29 - 30
11	Implement "N-Queens Problem" using Backtracking.	31 - 32
12	Leetcode exercises on Stacks, Queues, Circular Queues, Priority Queues.	33 - 43
13	Leetcode exercises on DFS, BFS	44 - 48
14	Leetcode exercises on Trees and Graphs	49 - 53

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate appropriate algorithm/design technique for solving given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 100

int adj[MAX][MAX];
int indegree[MAX];
int queue[MAX];
int n;
void topologicalSort()
{
    int front = 0, rear = -1;
    int count = 0;
    for(int i = 0; i < n; i++)
        indegree[i] = 0;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            if(adj[i][j] == 1)
                indegree[j]++;
    for(int i = 0; i < n; i++)
        if(indegree[i] == 0)
            queue[++rear] = i;
    printf("\nTopological Order: ");
    while(front <= rear)
    {
        int node = queue[front++];
        printf("%d ", node);
        count++;
        for(int i = 0; i < n; i++)
        {
            if(adj[node][i] == 1)
            {
                indegree[i]--;
                if(indegree[i] == 0)
                    queue[++rear] = i;
            }
        }
    }
    if(count != n)
        printf("\nGraph has a cycle! No topological ordering possible.\n");
}
```

```

int main()
{
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    clock_t start = clock();
    topologicalSort();
    clock_t end = clock();
    double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
    printf("\n\nTime Taken = %.3f ms\n", time_taken);
    return 0;
}

```

OUTPUT

```

C:\Users\student\Desktop\ts.exe
Enter number of vertices: 5
Enter adjacency matrix:
0 0 1 0 0
0 0 1 0 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0

Topological Order: 0 1 2 3 4

Time Taken = 1.000 ms

Process returned 0 (0x0)   execution time : 25.708 s
Press any key to continue.

```

Leetcode Problem 207- Course schedule

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int*
prerequisitesColSize)
{
    int indegree[numCourses];
    int adj[numCourses][numCourses];

    for(int i=0;i<numCourses;i++)
    {
        indegree[i]=0;
        for(int j=0;j<numCourses;j++)
            adj[i][j]=0;
    }

    for(int i=0;i<prerequisitesSize;i++)
    {
        int a=prerequisites[i][0];
        int b=prerequisites[i][1];

        adj[b][a]=1;
        indegree[a]++;
    }

    int queue[numCourses];
    int front=0,rear=-1;

    for(int i=0;i<numCourses;i++)
        if(indegree[i]==0)
            queue[++rear]=i;

    int count=0;

    while(front<=rear)
    {
        int u=queue[front++];
        count++;

        for(int v=0;v<numCourses;v++)
        {
            if(adj[u][v])
            {
                indegree[v]--;

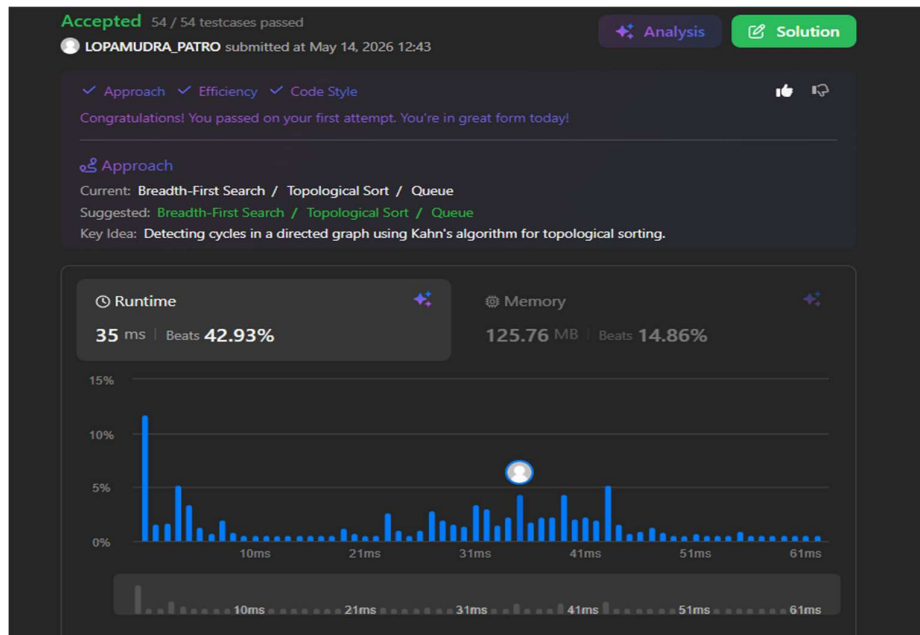
                if(indegree[v]==0)
                    queue[++rear]=v;
            }
        }
    }
}
```

```

    return count==numCourses;
}

```

OUTPUT



Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2

Input

numCourses =
2

prerequisites =
[[1,0],[0,1]]

Output

false

Expected

false

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
int a[10], dir[10];
void print(int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int mobile(int n) {
    int m = 0, pos = -1;

    for (int i = 0; i < n; i++) {
        int j = i + dir[i];

        if (j >= 0 && j < n && a[i] > a[j] && a[i] > m) {
            m = a[i];
            pos = i;
        }
    }
    return pos;
}

int main() {
    int n;
    printf("Enter number : ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = -1;
    }
    clock_t start = clock();
    print(n);
    while (1) {
        int i = mobile(n);
        if (i == -1)
            break;
        int j = i + dir[i];
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
        t = dir[i];
    }
}
```

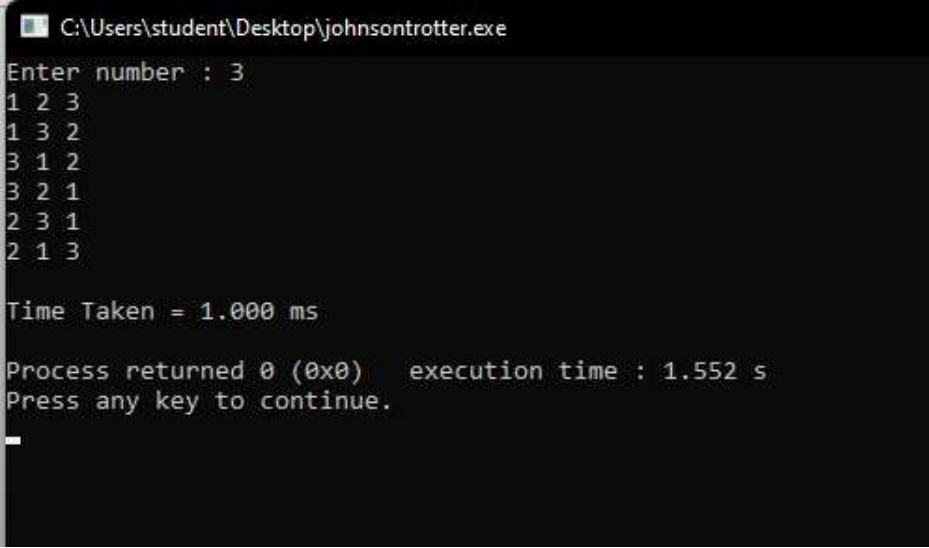


```

    dir[i] = dir[j];
    dir[j] = t;
    for (int k = 0; k < n; k++)
        if (a[k] > a[j])
            dir[k] *= -1;
    print(n);
}
clock_t end = clock();
double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("\nTime Taken = %.3f ms\n", time_taken);
return 0;
}

```

OUTPUT



```

C:\Users\student\Desktop\johnsontrotter.exe
Enter number : 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Time Taken = 1.000 ms

Process returned 0 (0x0)   execution time : 1.552 s
Press any key to continue.
_

```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

SOURCE CODE

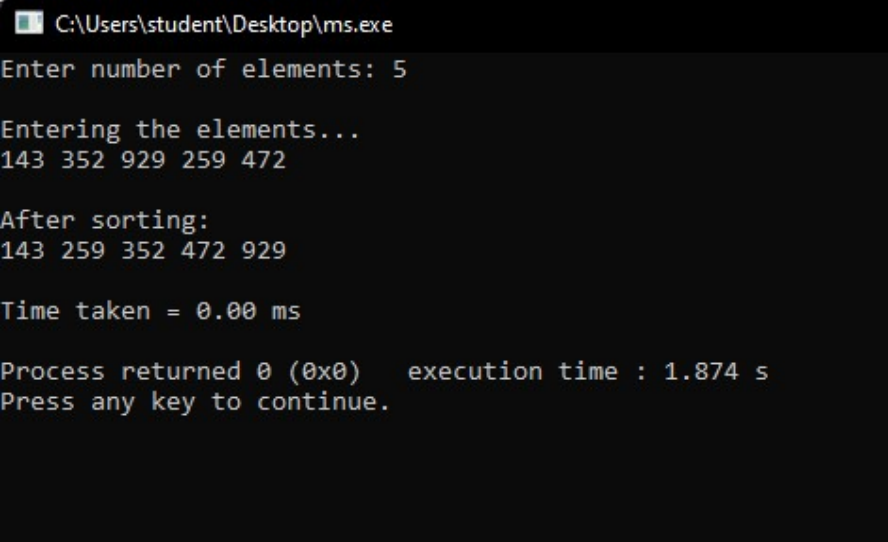
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int a[5000000], b[5000000];
void merge(int a[], int low, int mid, int high){
    int i = low, j = mid + 1, k = low;
    while(i <= mid && j <= high)
    {
        if(a[i] < a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }
    while(i <= mid)
        b[k++] = a[i++];
    while(j <= high)
        b[k++] = a[j++];
    for(int x = low; x <= high; x++)
        a[x] = b[x];
}
void merge_sort(int a[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;
        merge_sort(a, low, mid);
        merge_sort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    srand(time(NULL));
    printf("\nEnter the elements...\n");
    for(int i = 0; i < n; i++)
    {
        a[i] = rand() % 1000;
```

```

        printf("%d ", a[i]);
    }
    clock_t start_time = clock();
    merge_sort(a, 0, n - 1);
    clock_t end_time = clock();
    printf("\n\nAfter sorting:\n");
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    float total_time =
        (float)(end_time - start_time) * 1000 / CLOCKS_PER_SEC;
    printf("\n\nTime taken = %.2f ms\n", total_time);
    return 0;
}

```

OUTPUT



```

C:\Users\student\Desktop\ms.exe
Enter number of elements: 5

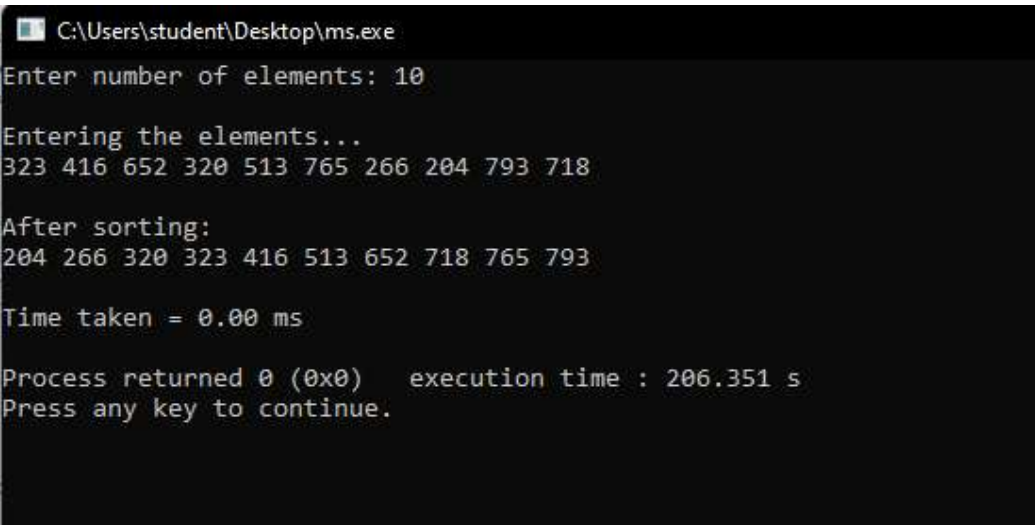
Entering the elements...
143 352 929 259 472

After sorting:
143 259 352 472 929

Time taken = 0.00 ms

Process returned 0 (0x0)   execution time : 1.874 s
Press any key to continue.

```



```

C:\Users\student\Desktop\ms.exe
Enter number of elements: 10

Entering the elements...
323 416 652 320 513 765 266 204 793 718

After sorting:
204 266 320 323 416 513 652 718 765 793

Time taken = 0.00 ms

Process returned 0 (0x0)   execution time : 206.351 s
Press any key to continue.

```

```
C:\Users\student\Desktop\ms.exe
Enter number of elements: 20

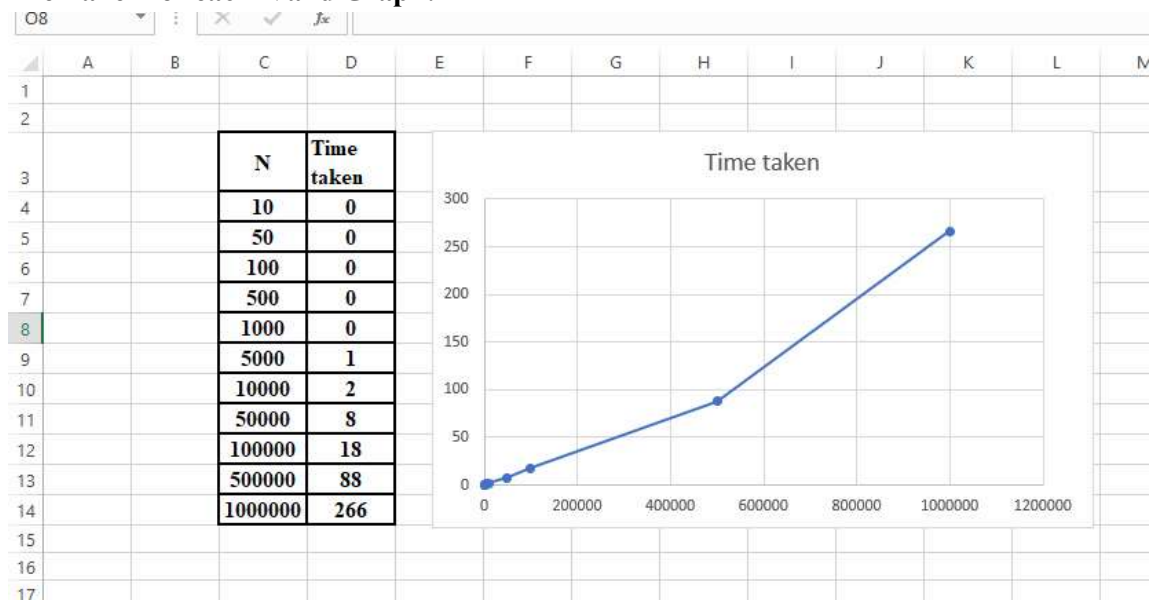
Entering the elements...
287 530 520 469 562 619 831 194 123 535 23 739 679 917 881 121 689 514 734 83

After sorting:
23 83 121 123 194 287 469 514 520 530 535 562 619 679 689 734 739 831 881 917

Time taken = 0.00 ms

Process returned 0 (0x0)   execution time : 3.295 s
Press any key to continue.
```

Time Taken for each N and Graph:



Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int a[5000000];
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int a[], int low, int high)
{
    int pivot = a[low];
    int i = low + 1;
    int j = high;
    while (i <= j)
    {
        while (i <= high && a[i] <= pivot)
            i++;
        while (j >= low && a[j] > pivot)
            j--;
        if (i < j)
            swap(&a[i], &a[j]);
    }
    swap(&a[low], &a[j]);
    return j;
}

void quick_sort(int a[], int low, int high)
{
    if (low < high)
    {
        int j = partition(a, low, high);
        quick_sort(a, low, j - 1);
        quick_sort(a, j + 1, high);
    }
}

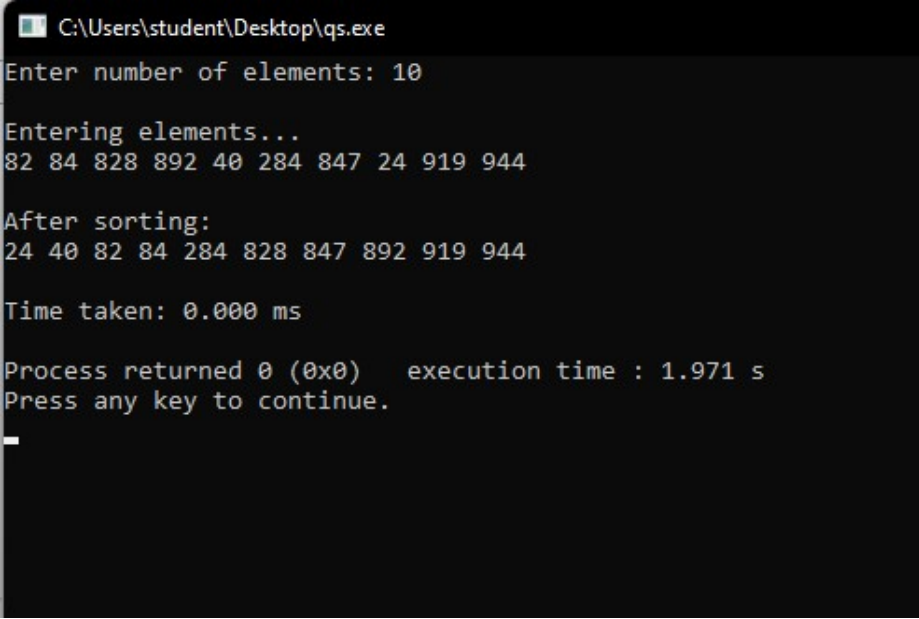
int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
```

```

srand(time(NULL));
printf("\nEntering elements...\n");
for (int i = 0; i < n; i++)
{
    a[i] = rand() % 1000;
    printf("%d ", a[i]);
}
clock_t start = clock();
quick_sort(a, 0, n - 1);
clock_t end = clock();
printf("\n\nAfter sorting:\n");
for (int i = 0; i < n; i++)
    printf("%d ", a[i]);
double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("\n\nTime taken: %.3f ms\n", time_taken);
return 0;
}

```

OUTPUT



```

C:\Users\student\Desktop\qs.exe
Enter number of elements: 10

Entering elements...
82 84 828 892 40 284 847 24 919 944

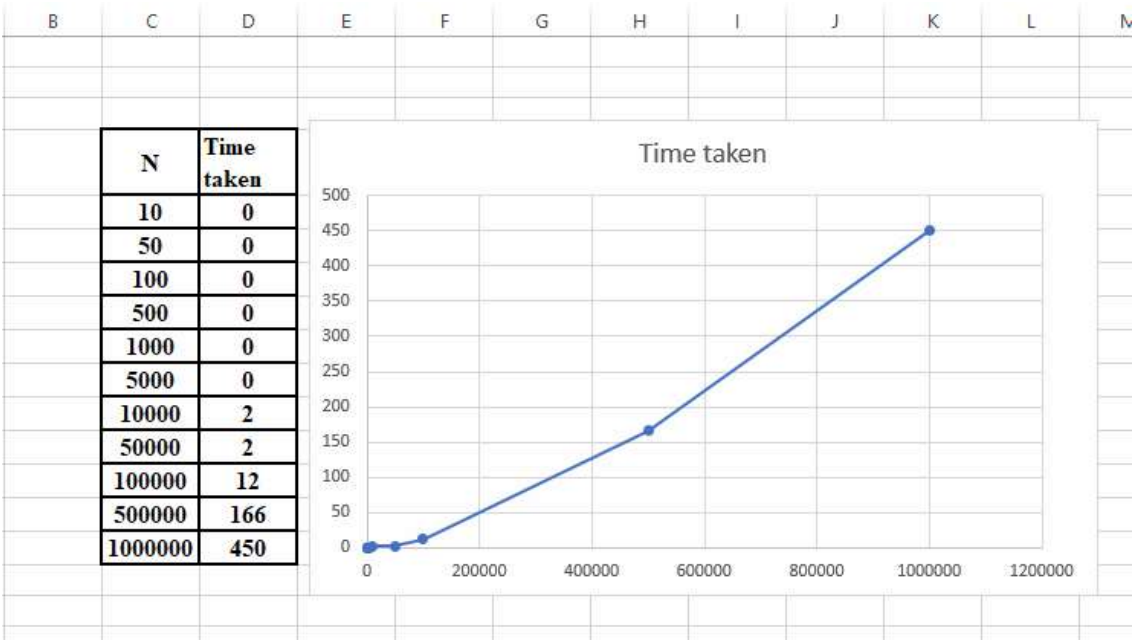
After sorting:
24 40 82 84 284 828 847 892 919 944

Time taken: 0.000 ms

Process returned 0 (0x0)   execution time : 1.971 s
Press any key to continue.

```

Time Taken for each N and Graph:



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int a[5000000];
void heapify(int a[], int i, int n)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i)
    {
        int temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;
        heapify(a, largest, n);
    }
}

void heap_sort(int a[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, i, n);
    for (int i = n - 1; i > 0; i--)
    {
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;
        heapify(a, 0, i);
    }
}

void print(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
```

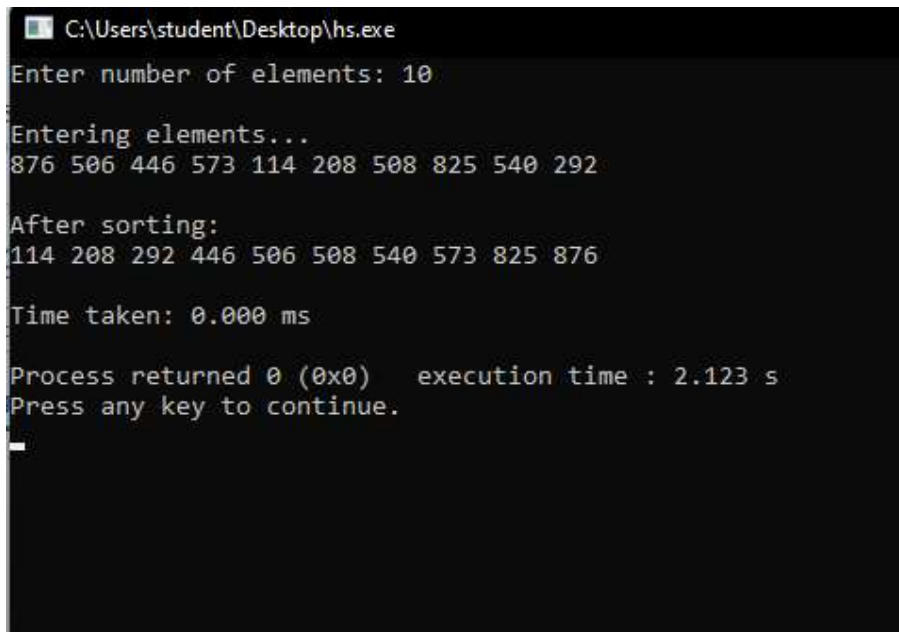


```

int main()
{
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    srand(time(NULL));
    printf("\nEnter elements...\n");
    for (int i = 0; i < n; i++)
        a[i] = rand() % 1000;
    print(a, n);
    clock_t start = clock();
    heap_sort(a, n);
    clock_t end = clock();
    printf("\nAfter sorting:\n");
    print(a, n);
    double time_taken = ((double)(end - start) / CLOCKS_PER_SEC) * 1000;
    printf("\nTime taken: %.3f ms\n", time_taken);
    return 0;
}

```

OUTPUT



```

C:\Users\student\Desktop\hs.exe
Enter number of elements: 10

Entering elements...
876 506 446 573 114 208 508 825 540 292

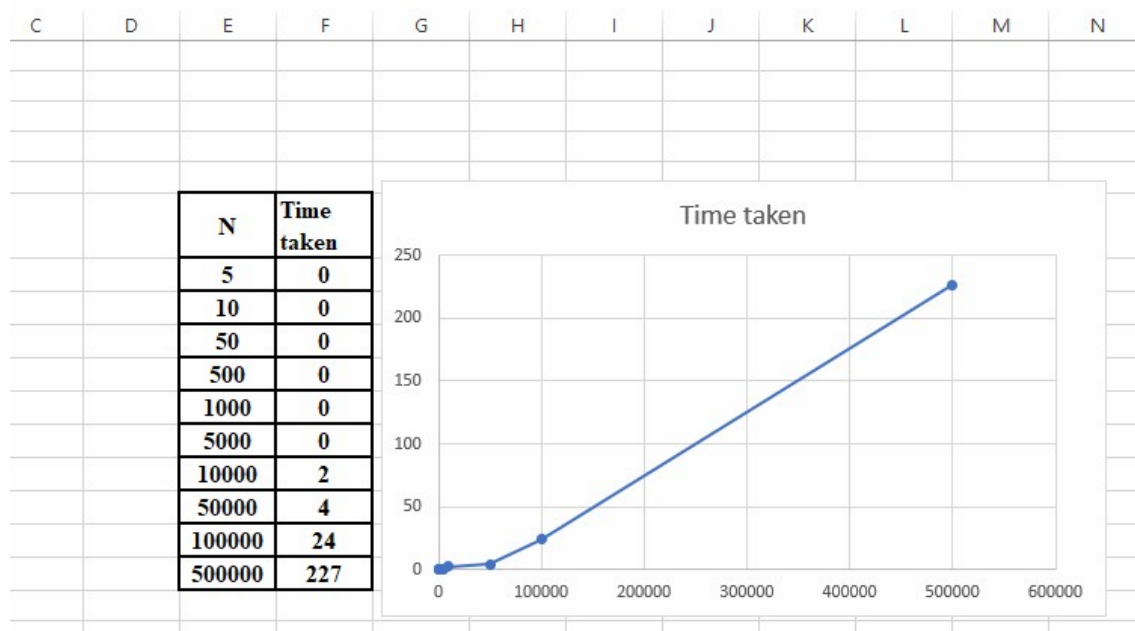
After sorting:
114 208 292 446 506 508 540 573 825 876

Time taken: 0.000 ms

Process returned 0 (0x0)   execution time : 2.123 s
Press any key to continue.
_

```

Time Taken for each N and Graph:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
int p[50],w[50],n,m;
void inputs(){
    printf("\nEnter number of items: ");
    scanf("%d",&n);
    printf("\nEnter capacity: ");
    scanf("%d",&m);
    printf("\nEnter weights: ");
    for(int i=1;i<=n;i++)
        scanf("%d",&w[i]);
    printf("\nEnter profits: ");
    for(int i=1;i<=n;i++)
        scanf("%d",&p[i]);
}

int max(int a,int b){
    return (a>b)?a:b;
}

int knapsack(){
    int v[n+1][m+1];
    for(int i=0;i<=n;i++){
        for(int j=0;j<=m;j++){
            if(i==0||j==0)
                v[i][j]=0;
            else if(w[i]>j)
                v[i][j]=v[i-1][j];
            else
                v[i][j]=max(v[i-1][j],v[i-1][j-w[i]]+p[i]);
        }
    }
    printf("\nDP Table:\n")
    for(int i=0;i<=n;i++){
        for(int j=0;j<=m;j++)
            printf("%4d",v[i][j]);
        printf("\n");
    }
    printf("\nOptimal Profit: %d",v[n][m]);
}

int main(){
    inputs();
    clock_t start=clock();
```

```

    knapsack();
    clock_t end=clock();
    float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
    printf("\nTime taken: %.3f ms",time_taken);
    return 0;
}

```

OUTPUT

```

C:\Users\student\Desktop\knapsackd.exe

Enter number of items: 4

Enter capacity: 8

Enter weights: 2 3 1 4

Enter profits: 3 4 6 5

DP Table:
  0  0  0  0  0  0  0  0  0
  0  0  3  3  3  3  3  3  3
  0  0  3  4  4  7  7  7  7
  0  6  6  9 10 10 13 13 13
  0  6  6  9 10 11 13 14 15

Optimal Profit: 15
Time taken: 1.000 ms
Process returned 0 (0x0)   execution time : 27.362 s
Press any key to continue.

```

Leetcode Problem 70 - Climbing Stairs

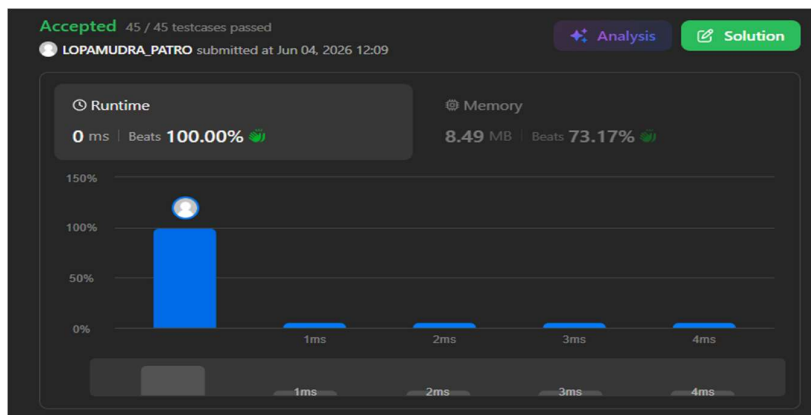
```
int climbStairs(int n)
{
    if(n <= 2)
        return n;

    int a = 1, b = 2, c;

    for(int i = 3; i <= n; i++)
    {
        c = a + b;
        a = b;
        b = c;
    }

    return b;
}
```

OUTPUT



Accepted Runtime: 0 ms

Case 1 Case 2

Input

n = 2

Output

2

Expected

2

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n = 3

Output

3

Expected

3

Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
#define INF 999
int main() {
    int n;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int graph[n][n];
    printf("Enter adjacency matrix:\n");
    printf("Enter 999 if no edge exists between vertices\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    int d[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            d[i][j] = graph[i][j];
    clock_t start = clock();
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                if (d[j][i] + d[i][k] < d[j][k]) {
                    d[j][k] = d[j][i] + d[i][k];
                }
            }
        }
    }
    clock_t end = clock();
    printf("Shortest distance matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", d[i][j]);
        }
        printf("\n");
    }
    float time_taken = ((float)(end - start) / CLOCKS_PER_SEC) * 1000;
    printf("\nTime taken: %.3f ms", time_taken);
    return 0;
}
```

OUTPUT

```
C:\Users\student\Desktop\floyd.exe
Enter number of vertices: 4
Enter adjacency matrix:
Enter 999 if no edge exists between vertices
0 999 3 999
2 0 999 999
999 7 0 1
6 999 999 0
Shortest distance matrix:
0 10 3 4
2 0 5 6
7 7 0 1
6 16 9 0

Time taken: 0.000 ms
Process returned 0 (0x0)   execution time : 33.464 s
Press any key to continue.
```

Leetcode Problem 1334 - Find the City With the Smallest Number of Neighbors at a Threshold Distance

```
#define INF 1000000
int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold)
{
    int dist[n][n];
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            if(i == j)
                dist[i][j] = 0;
            else
                dist[i][j] = INF;
        }
    }
    for(int i = 0; i < edgesSize; i++)
    {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];
        dist[u][v] = w;
        dist[v][u] = w;
    }
    for(int k = 0; k < n; k++)
    {
        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if(dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }
    int ans = -1;
    int minReach = INF;
    for(int i = 0; i < n; i++)
    {
        int count = 0;
        for(int j = 0; j < n; j++)
        {
            if(i != j && dist[i][j] <= distanceThreshold)
                count++;
        }
        if(count <= minReach)
        {
            minReach = count;
        }
    }
}
```

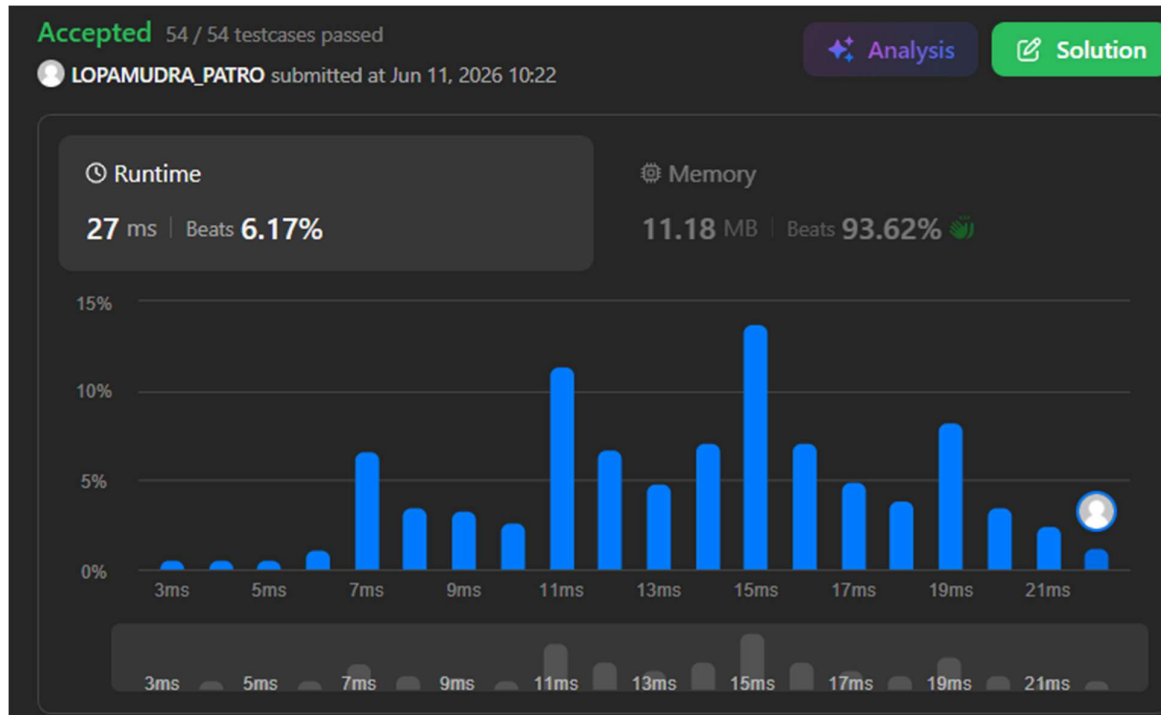


```

        ans = i;
    }
}
return ans;
}

```

OUTPUT



Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

n =
4

edges =
[[0, 1, 3], [1, 2, 1], [1, 3, 4], [2, 3, 1]]

distanceThreshold =
4

Output
3

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

n =
5

edges =
[[0, 1, 2], [0, 4, 8], [1, 2, 3], [1, 4, 2], [2, 3, 1], [3, 4, 1]]

distanceThreshold =
2

Output
0

Lab program 8:

(i) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
void prims(int n,int graph[n][n]){
    int visited[n];
    for(int i=0;i<n;i++){
        visited[i]=0;
    }
    int edges=0;
    visited[0]=1;
    int cost=0;
    printf("\nEdge\tWeight\n");
    while(edges<n-1){
        int min=100,x=-1,y=-1;
        for(int i=0;i<n;i++){
            if(visited[i]==1){
                for(int j=0;j<n;j++){
                    if(visited[j]==0 && graph[i][j]!=0){
                        if(graph[i][j]<min){
                            min=graph[i][j];
                            x=i;
                            y=j;
                        }
                    }
                }
            }
        }

        printf("%d - %d\t%d\n",x,y,graph[x][y]);
        cost+=graph[x][y];
        visited[y]=1;
        edges++;
    }

    printf("\nMinimum Cost %d\n",cost);
}

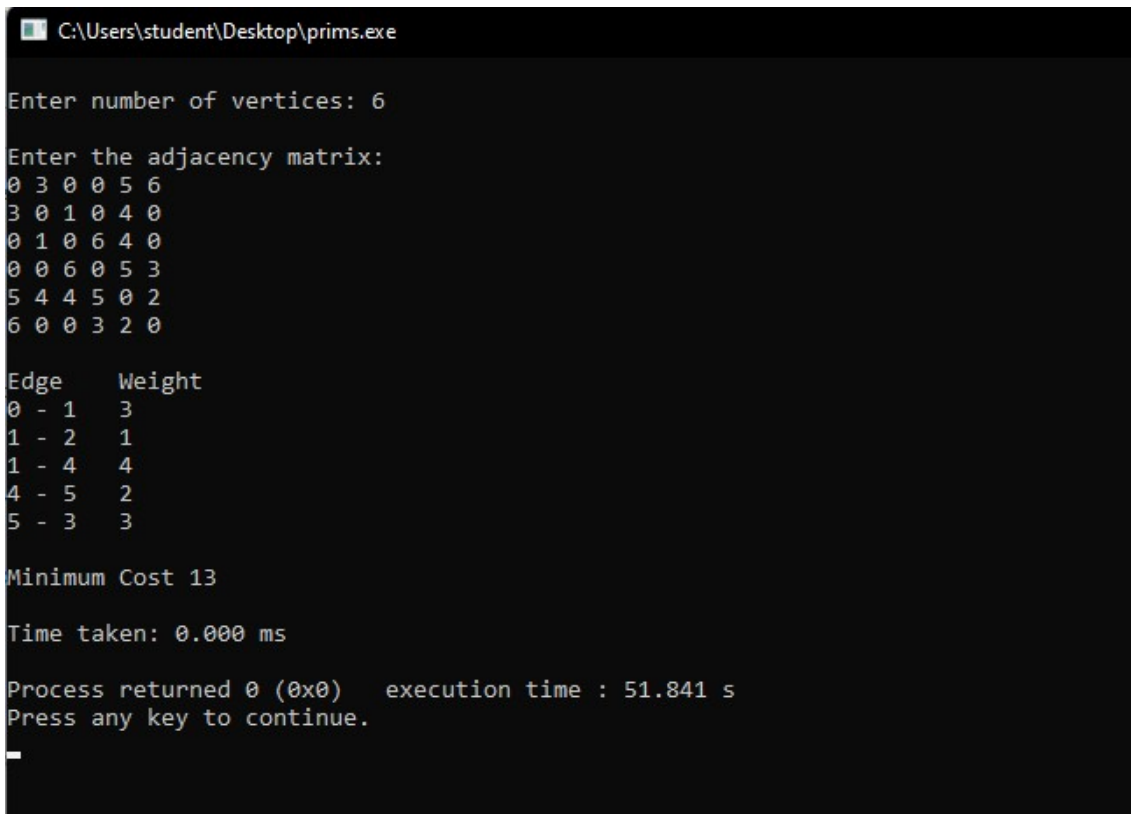
int main(){
    int n;
    printf("\nEnter number of vertices: ");
    scanf("%d",&n);
    int graph[n][n];
    printf("\nEnter the adjacency matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
```

```

        scanf("%d",&graph[i][j]);
    }
}
clock_t start=clock();
prims(n,graph);
clock_t end=clock();
float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
printf("\nTime taken: %.3f ms\n",time_taken);
return 0;
}

```

OUTPUT



```

C:\Users\student\Desktop\prims.exe

Enter number of vertices: 6

Enter the adjacency matrix:
0 3 0 0 5 6
3 0 1 0 4 0
0 1 0 6 4 0
0 0 6 0 5 3
5 4 4 5 0 2
6 0 0 3 2 0

Edge    Weight
0 - 1    3
1 - 2    1
1 - 4    4
4 - 5    2
5 - 3    3

Minimum Cost 13

Time taken: 0.000 ms

Process returned 0 (0x0)   execution time : 51.841 s
Press any key to continue.
_

```

Lab program 8:

(ii) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
int parent[20];
int find(int i)
{
    while(parent[i] != -1)
        i = parent[i];
    return i;
}
void uni(int a, int b)
{
    parent[b] = a;
}
int main()
{
    int n, cost[20][20];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if(cost[i][j] == 0)
                cost[i][j] = 999;
        }
    }
    for(int i = 0; i < n; i++)
        parent[i] = -1;
    clock_t start = clock();
    int edges = 0, mincost = 0;
    printf("\nEdge\tWeight\n");
    while(edges < n - 1)
    {
        int min = 999;
        int a = -1, b = -1;
        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < n; j++)
            {
                if(cost[i][j] < min)
                {
                    min = cost[i][j];
                    a = i;
                }
            }
        }
        uni(a, b);
        edges++;
        mincost += min;
    }
    clock_t end = clock();
    printf("\nTime taken: %f\n", (double)(end - start) / CLOCKS_PER_SEC);
    printf("\nMinimum Cost Spanning Tree: \n");
    for(int i = 0; i < n; i++)
        printf("%d ", parent[i]);
    printf("\n");
}
```

```

        b = j;
    }
}

int u = find(a);
int v = find(b);
if(u != v)
{
    uni(u, v);
    printf("%d - %d\t%d\n", a, b, min);
    mincost += min;
    edges++;
}
cost[a][b] = cost[b][a] = 999;
}
clock_t end = clock();
printf("\nMinimum Cost = %d\n", mincost);
float time_taken = ((float)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("Time Taken = %.3f ms\n", time_taken);
return 0;
}

```

OUTPUT

```

Enter number of vertices: 6
Enter adjacency matrix:
0 3 0 1 0 0
3 0 1 3 0 0
0 1 0 1 5 4
1 3 1 0 6 0
0 0 5 6 0 2
0 0 4 0 2 0

Edge    Weight
0 - 3    1
1 - 2    1
2 - 3    1
4 - 5    2
2 - 5    4

Minimum Cost = 9
Time Taken = 1.000 ms

Process returned 0 (0x0)   execution time : 62.866 s
Press any key to continue.

```

Lab program 9:

Implement Fractional Knapsack using Greedy technique.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
int main()
{
    int n,m;
    printf("Enter number of items: ");
    scanf("%d",&n);
    int p[n],w[n];
    float r[n],x[n];
    printf("Enter profits: ");
    for(int i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter weights: ");
    for(int i=0;i<n;i++)
    {
        scanf("%d",&w[i]);
        r[i]=(float)p[i]/w[i];
    }
    printf("Enter capacity: ");
    scanf("%d",&m);
    clock_t start=clock();
    for(int i=0;i<n-1;i++)
    {
        for(int j=0;j<n-i-1;j++)
        {
            if(r[j]<r[j+1])
            {
                float tr=r[j];
                r[j]=r[j+1];
                r[j+1]=tr;
                int tp=p[j];
                p[j]=p[j+1];
                p[j+1]=tp;
                int tw=w[j];
                w[j]=w[j+1];
                w[j+1]=tw;
            }
        }
    }
    for(int i=0;i<n;i++)
        x[i]=0;
    float profit=0;
    int rem=m;
    for(int i=0;i<n;i++)
```

```

{
    if(w[i]<=rem)
    {
        x[i]=1;
        profit+=p[i];
        rem-=w[i];
    }
    else
    {
        x[i]=(float)rem/w[i];
        profit+=x[i]*p[i];
        break;
    }
}
clock_t end=clock();
printf("\nProfit\tWeight\tRatio\tFraction\n");
for(int i=0;i<n;i++)
    printf("%d\t%d\t%.2f\t%.2f\n",
        p[i],w[i],r[i],x[i]);
printf("\nMaximum Profit = %.2f\n",profit);
float time_taken=((float)(end-start))/CLOCKS_PER_SEC*1000;
printf("Time Taken = %.3f ms\n",time_taken);
return 0;
}

```

OUTPUT

```

Enter number of items: 5
Enter profits: 25 20 15 40 50
Enter weights: 3 2 1 4 5
Enter capacity: 6

Profit  Weight  Ratio  Fraction
15      1       15.00   1.00
20      2       10.00   1.00
40      4       10.00   0.75
50      5       10.00   0.00
25      3       8.33    0.00

Maximum Profit = 65.00
Time Taken = 0.000 ms

Process returned 0 (0x0)   execution time : 20.961 s
Press any key to continue.

```

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

SOURCE CODE

```
#include <stdio.h>
#include <time.h>
#define INF 999
int minDistance(int dist[], int visited[], int n)
{
    int min = INF, min_index = -1;
    for(int i = 0; i < n; i++)
    {
        if(!visited[i] && dist[i] < min)
        {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

int main()
{
    int n, src;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    int cost[n][n];
    printf("Enter adjacency matrix (0 if no edge):\n");
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &cost[i][j]);
            if(i != j && cost[i][j] == 0)
                cost[i][j] = INF;
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &src);
    int dist[n], visited[n];
    clock_t start = clock();
    for(int i = 0; i < n; i++)
    {
        dist[i] = cost[src][i];
        visited[i] = 0;
    }
    dist[src] = 0;
    visited[src] = 1;
```



```

for(int i = 1; i < n; i++)
{
    int u = minDistance(dist, visited, n);
    if(u == -1)
        break;
    visited[u] = 1;
    for(int v = 0; v < n; v++)
    {
        if(!visited[v] &&
            cost[u][v] != INF &&
            dist[u] + cost[u][v] < dist[v])
        {
            dist[v] = dist[u] + cost[u][v];
        }
    }
}
clock_t end = clock();
printf("\nVertex\tDistance\n");
for(int i = 0; i < n; i++)
    printf("%d\t%d\n", i, dist[i]);
float time_taken =
    ((float)(end - start) / CLOCKS_PER_SEC) * 1000;
printf("\nTime Taken: %.3f ms\n", time_taken);
return 0;
}

```

OUTPUT

```

Enter number of vertices: 6
Enter adjacency matrix (0 if no edge):
0 15 10 0 45 0
0 0 15 0 20 0
20 0 0 20 0 0
0 10 0 0 35 0
0 0 0 30 0 0
0 0 0 4 0 0
Enter source vertex: 5

Vertex  Distance
0       49
1       14
2       29
3        4
4       34
5        0

Time Taken: 0.000 ms

Process returned 0 (0x0)   execution time : 69.873 s
Press any key to continue.
_

```

Lab program 11:

Implement “N-Queens Problem” using Backtracking.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int x[20], n, count = 0;
int place(int k, int i)
{
    for(int j = 1; j < k; j++)
    {
        if(x[j] == i || abs(x[j] - i) == abs(j - k))
            return 0;
    }
    return 1;
}

void print()
{
    count++;
    printf("\nSolution %d:\n", count);
    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
        {
            if(x[i] == j)
                printf("Q ");
            else
                printf(". ");
        }
        printf("\n");
    }
}

void nqueen(int k)
{
    for(int i = 1; i <= n; i++)
    {
        if(place(k, i))
        {
            x[k] = i;
            if(k == n)
                print();
            else
                nqueen(k + 1);
        }
    }
}
```

```

int main()
{
    printf("Enter value of N: ");
    scanf("%d", &n);
    clock_t start = clock();
    nqueen(1);
    clock_t end = clock();
    if(count == 0)
        printf("\nNo solution exists\n");
    printf("\nTotal Solutions = %d\n", count);
    float time_taken =
        ((float)(end - start) / CLOCKS_PER_SEC) * 1000;
    printf("Time Taken = %.3f ms\n", time_taken);
    return 0;
}

```

OUTPUT

```

Enter value of N: 4

Solution 1:
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2:
. . Q .
Q . . .
. . . Q
. Q . .

Total Solutions = 2
Time Taken = 1.000 ms

Process returned 0 (0x0)   execution time : 2.297 s
Press any key to continue.

```

Leetcode exercises on Stacks, Queues, Circular Queues, Priority Queues.

Leetcode Problem 225. Implement Stack using Queues (Stack)

```
#include <stdlib.h>
#include <stdbool.h>

#define MAX 100

typedef struct {
    int data[MAX];
    int front, rear;
} Queue;

typedef struct {
    Queue q1;
    Queue q2;
} MyStack;

void initQueue(Queue *q) {
    q->front = 0;
    q->rear = -1;
}

bool isEmpty(Queue *q) {
    return q->front > q->rear;
}

void enqueue(Queue *q, int x) {
    q->data[++q->rear] = x;
}

int dequeue(Queue *q) {
    return q->data[q->front++];
}

int frontQueue(Queue *q) {
    return q->data[q->front];
}

MyStack* myStackCreate() {
    MyStack *s = (MyStack*)malloc(sizeof(MyStack));
    initQueue(&s->q1);
    initQueue(&s->q2);
    return s;
}
```

```

void myStackPush(MyStack* obj, int x) {
    enqueue(&obj->q2, x);

    while (!isEmpty(&obj->q1)) {
        enqueue(&obj->q2, dequeue(&obj->q1));
    }

    Queue temp = obj->q1;
    obj->q1 = obj->q2;
    obj->q2 = temp;
}

int myStackPop(MyStack* obj) {
    return dequeue(&obj->q1);
}

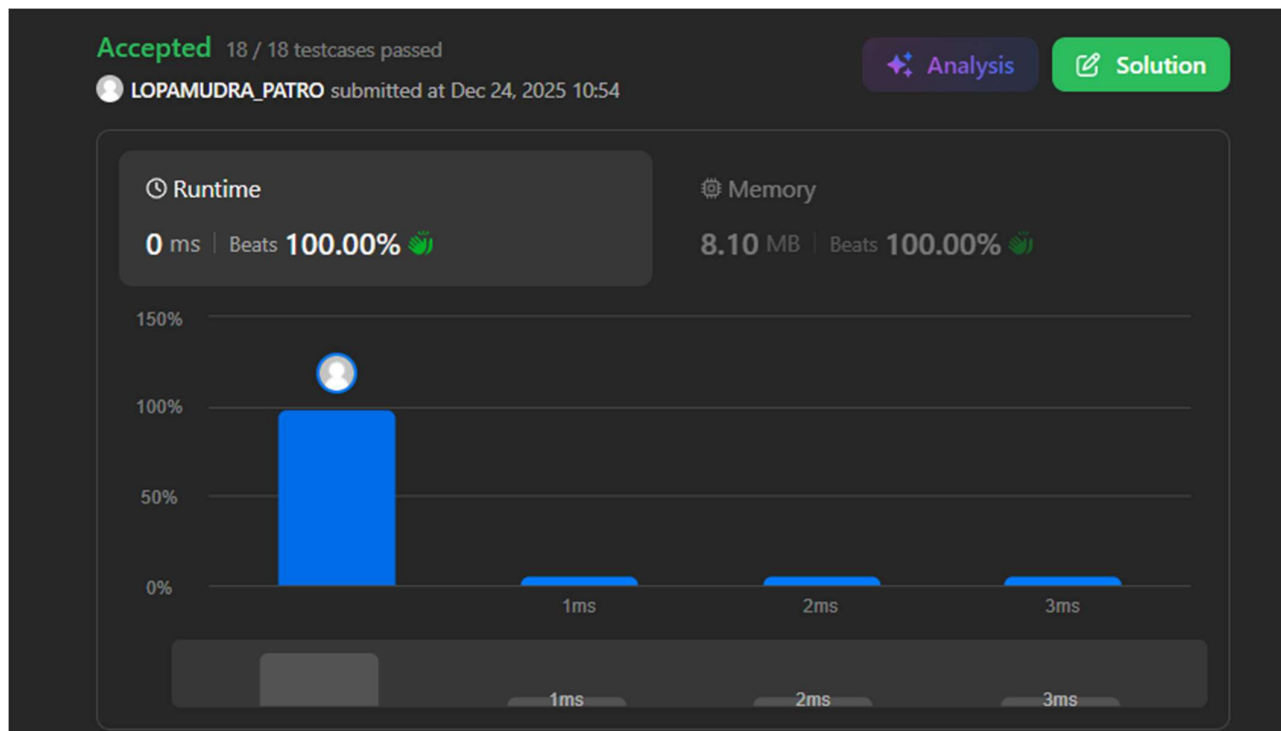
int myStackTop(MyStack* obj) {
    return frontQueue(&obj->q1);
}

bool myStackEmpty(MyStack* obj) {
    return isEmpty(&obj->q1);
}

void myStackFree(MyStack* obj) {
    free(obj);
}

```

OUTPUT



Testcase | **Test Result**

Accepted Runtime: 0 ms

☒ Case 1

Input

```
["MyStack","push","push","top","pop","empty"]
```

```
[[],[1],[2],[],[],[1]]
```

Output

```
[null,null,null,2,2,false]
```

Expected

```
[null,null,null,2,2,false]
```

Leetcode Problem 232. Implement Queue Using Stacks (Queues)

```
#include <stdlib.h>
#include <stdbool.h>

#define MAX 100

typedef struct {
    int data[MAX];
    int top;
} Stack;

typedef struct {
    Stack s1; // input stack
    Stack s2; // output stack
} MyQueue;

void initStack(Stack *s) {
    s->top = -1;
}

bool isStackEmpty(Stack *s) {
    return s->top == -1;
}

void pushStack(Stack *s, int x) {
    s->data[++s->top] = x;
}

int popStack(Stack *s) {
    return s->data[s->top--];
}

int peekStack(Stack *s) {
    return s->data[s->top];
}

MyQueue* myQueueCreate() {
    MyQueue *q = (MyQueue*)malloc(sizeof(MyQueue));
    initStack(&q->s1);
    initStack(&q->s2);
    return q;
}

void myQueuePush(MyQueue* obj, int x) {
    pushStack(&obj->s1, x);
}

int myQueuePop(MyQueue* obj) {
```

```

    if (isStackEmpty(&obj->s2)) {
        while (!isStackEmpty(&obj->s1)) {
            pushStack(&obj->s2, popStack(&obj->s1));
        }
    }
    return popStack(&obj->s2);
}

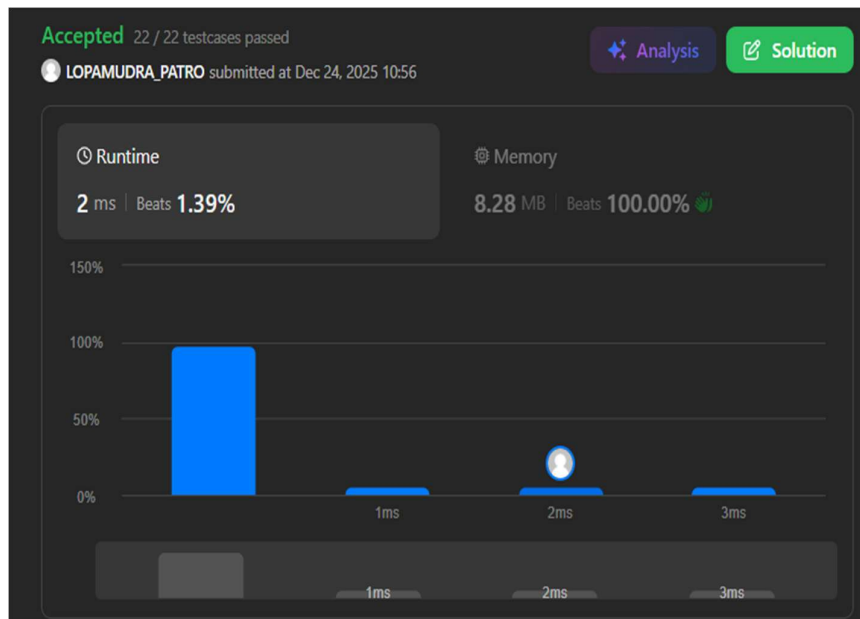
int myQueuePeek(MyQueue* obj) {
    if (isStackEmpty(&obj->s2)) {
        while (!isStackEmpty(&obj->s1)) {
            pushStack(&obj->s2, popStack(&obj->s1));
        }
    }
    return peekStack(&obj->s2);
}

bool myQueueEmpty(MyQueue* obj) {
    return isStackEmpty(&obj->s1) && isStackEmpty(&obj->s2);
}

void myQueueFree(MyQueue* obj) {
    free(obj);
}

```

OUTPUT



☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1

Input

`["MyQueue","push","push","peek","pop","empty"]`

`[[],[1],[2],[1],[],[1]]`

Output

`[null,null,null,1,1,false]`

Expected

`[null,null,null,1,1,false]`

Leetcode Problem 622. Design Circular Queue (Circular Queues)

```
#include <stdlib.h>
#include <stdbool.h>

typedef struct {
    int *arr;
    int front;
    int rear;
    int size;
    int capacity;
} MyCircularQueue;

MyCircularQueue* myCircularQueueCreate(int k) {
    MyCircularQueue* q = (MyCircularQueue*)malloc(sizeof(MyCircularQueue));
    q->capacity = k;
    q->arr = (int*)malloc(k * sizeof(int));
    q->front = 0;
    q->rear = -1;
    q->size = 0;
    return q;
}

bool myCircularQueueEnQueue(MyCircularQueue* q, int value) {
    if (q->size == q->capacity)
        return false;

    q->rear = (q->rear + 1) % q->capacity;
    q->arr[q->rear] = value;
    q->size++;
    return true;
}

bool myCircularQueueDeQueue(MyCircularQueue* q) {
    if (q->size == 0)
        return false;

    q->front = (q->front + 1) % q->capacity;
    q->size--;
    return true;
}

int myCircularQueueFront(MyCircularQueue* q) {
    if (q->size == 0)
        return -1;
    return q->arr[q->front];
}

int myCircularQueueRear(MyCircularQueue* q) {
    if (q->size == 0)
```

```

        return -1;
    return q->arr[q->rear];
}

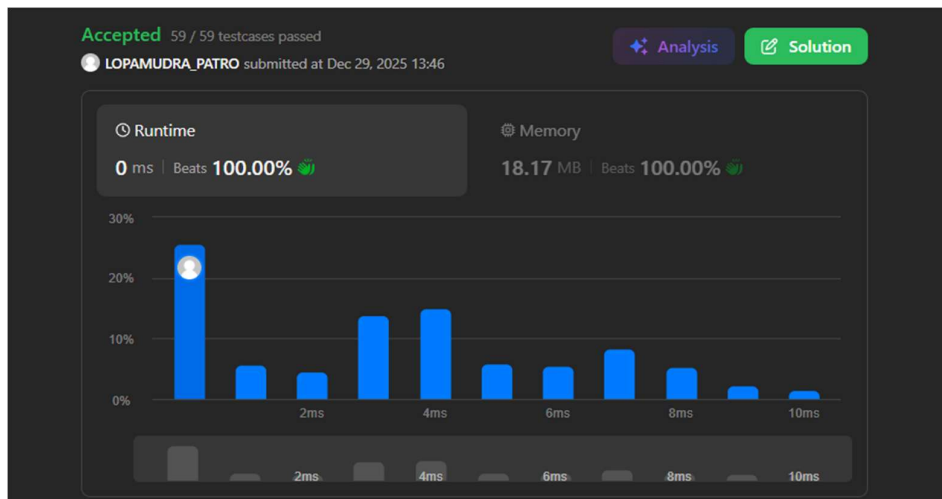
bool myCircularQueueIsEmpty(MyCircularQueue* q) {
    return q->size == 0;
}

bool myCircularQueueIsFull(MyCircularQueue* q) {
    return q->size == q->capacity;
}

void myCircularQueueFree(MyCircularQueue* q) {
    free(q->arr);
    free(q);
}

```

OUTPUT



Accepted Runtime: 0 ms

☒ Case 1

Input

```
["MyCircularQueue", "enqueue", "enqueue", "enqueue", "enqueue", "Rear", "isFull", "deQueue", "enqueue", "Rear"]
```

```
[[3], [1], [2], [3], [4], [], [], [], [4], []]
```

Output

```
[null, true, true, true, false, 3, true, true, true, 4]
```

Expected

```
[null, true, true, true, false, 3, true, true, true, 4]
```

Leetcode Problem 743. Network Delay Time (Priority Queues)

```
#include <stdlib.h>
#include <limits.h>

int networkDelayTime(int** times, int timesSize, int* timesColSize, int n, int k) {

    // Create adjacency matrix
    int graph[101][101];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            graph[i][j] = INT_MAX;
        }
    }

    for (int i = 0; i < timesSize; i++) {
        int u = times[i][0];
        int v = times[i][1];
        int w = times[i][2];

        graph[u][v] = w;
    }

    int dist[101];
    int visited[101];

    for (int i = 1; i <= n; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }

    dist[k] = 0;

    for (int i = 1; i <= n; i++) {

        int u = -1;
```

```

for (int j = 1; j <= n; j++) {
    if (!visited[j] &&
        (u == -1 || dist[j] < dist[u])) {
        u = j;
    }
}

if (dist[u] == INT_MAX)
    break;

visited[u] = 1;

for (int v = 1; v <= n; v++) {

    if (graph[u][v] != INT_MAX &&
        dist[u] + graph[u][v] < dist[v]) {

        dist[v] = dist[u] + graph[u][v];
    }
}

int ans = 0;

for (int i = 1; i <= n; i++) {

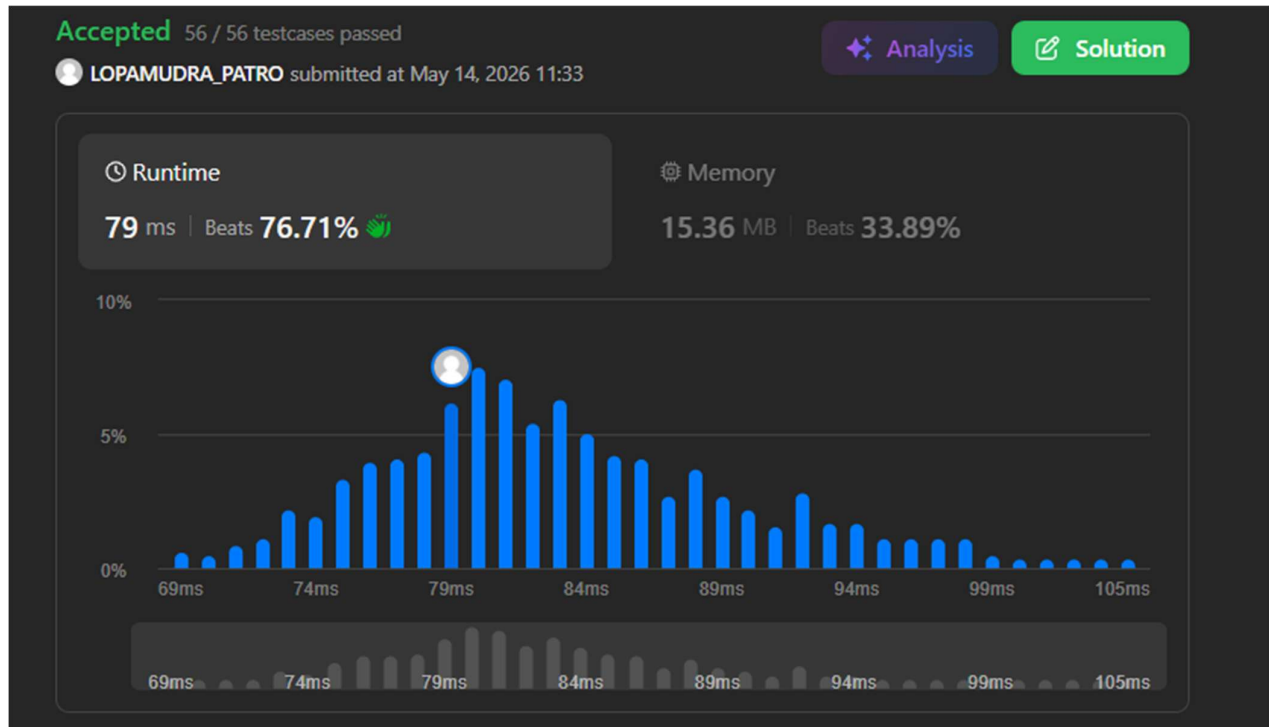
    if (dist[i] == INT_MAX)
        return -1;

    if (dist[i] > ans)
        ans = dist[i];
}

return ans;
}

```

OUTPUT



☒ Testcase | [Test Result](#)

Case 1 Case 2 Case 3

times =

`[[2,1,1], [2,3,1], [3,4,1]]`

n =

4

k =

2

Case 1 Case 2

times =

`[[1,2,1]]`

n =

2

k =

1

☒ Testcase | [Test Result](#)

Case 1 Case 2 Case 3

times =

`[[1,2,1]]`

n =

2

k =

2

Leetcode exercises on DFS, BFS

Leetcode Problem 847. Shortest Path Visiting All Nodes (BFS)

```
#include <stdlib.h>
#include <stdbool.h>

typedef struct {
    int node;
    int mask;
    int dist;
} State;

int shortestPathLength(int** graph, int graphSize, int* graphColSize) {

    int finalMask = (1 << graphSize) - 1;
    bool visited[12][4096] = {false};

    State queue[50000];

    int front = 0;
    int rear = 0;

    for (int i = 0; i < graphSize; i++) {

        int mask = (1 << i);

        queue[rear++] = (State){i, mask, 0};

        visited[i][mask] = true;
    }

    while (front < rear) {

        State curr = queue[front++];

        int node = curr.node;
        int mask = curr.mask;
        int dist = curr.dist;
        if (mask == finalMask)
            return dist;

        for (int i = 0; i < graphColSize[node]; i++) {

            int next = graph[node][i];

            int nextMask = mask | (1 << next);

            if (!visited[next][nextMask]) {
```

```

        visited[next][nextMask] = true;

        queue[rear++] = (State){
            next,
            nextMask,
            dist + 1
        };
    }
}

return -1;
}

```

OUTPUT



Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

graph =

[[1,2,3], [0], [0], [0]]

Output

4

Expected

4

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

graph =

[[1], [0,2,4], [1,3,4], [2], [1,2]]

Output

4

Expected

4

Leetcode 3558. Number of Ways to Assign Edge Weights I (DFS)

```
#define MOD 1000000007LL

long long power(long long a, long long b) {
    long long res = 1;

    while (b > 0) {
        if (b & 1)
            res = (res * a) % MOD;

        a = (a * a) % MOD;
        b >>= 1;
    }

    return res;
}

int assignEdgeWeights(int** edges, int edgesSize, int* edgesColSize) {
    int n = edgesSize + 1;

    int degree[n + 1];
    for (int i = 0; i <= n; i++)
        degree[i] = 0;

    for (int i = 0; i < edgesSize; i++) {
        degree[edges[i][0]]++;
        degree[edges[i][1]]++;
    }

    int* adj[n + 1];
    for (int i = 1; i <= n; i++)
        adj[i] = (int*)malloc(degree[i] * sizeof(int));

    int idx[n + 1];
    for (int i = 0; i <= n; i++)
        idx[i] = 0;

    for (int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];

        adj[u][idx[u]++] = v;
        adj[v][idx[v]++] = u;
    }

    int q[n + 1];
    int depth[n + 1];
    int vis[n + 1];
}
```

```

for (int i = 0; i <= n; i++)
    vis[i] = 0;

int front = 0, rear = 0;

q[rear++] = 1;
vis[1] = 1;
depth[1] = 0;

int maxDepth = 0;

while (front < rear) {
    int node = q[front++];

    if (depth[node] > maxDepth)
        maxDepth = depth[node];

    for (int i = 0; i < degree[node]; i++) {
        int nei = adj[node][i];

        if (!vis[nei]) {
            vis[nei] = 1;
            depth[nei] = depth[node] + 1;
            q[rear++] = nei;
        }
    }
}

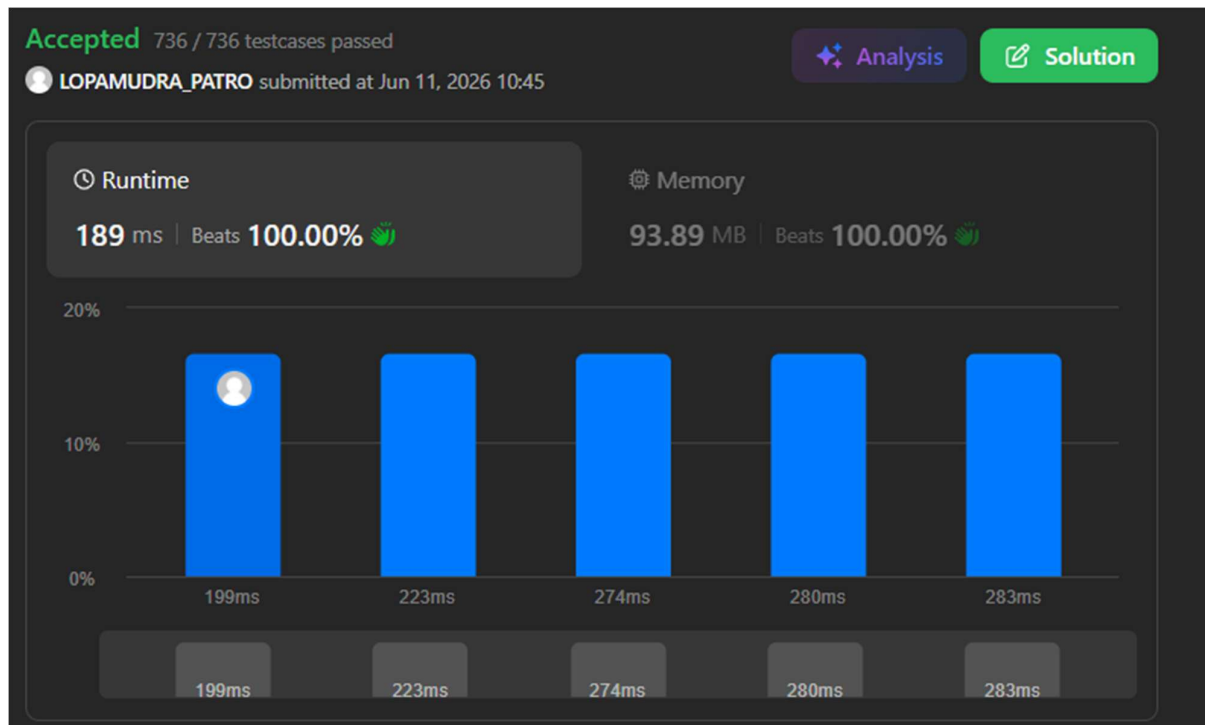
for (int i = 1; i <= n; i++)
    free(adj[i]);

if (maxDepth == 0)
    return 0;

return (int)power(2, maxDepth - 1);
}

```

OUTPUT



☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

edges =
[[1,2]]

Output

1

Expected

1

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

edges =
[[1,2], [1,3], [3,4], [3,5]]

Output

2

Expected

2

Leetcode exercises on Trees and Graphs

Leetcode 493. Reverse Pairs (Trees)

```
#include <stdlib.h>

int mergeSort(int* nums, int left, int right);
int merge(int* nums, int left, int mid, int right);

int reversePairs(int* nums, int numsSize) {
    if (numsSize == 0)
        return 0;

    return mergeSort(nums, 0, numsSize - 1);
}

int mergeSort(int* nums, int left, int right) {
    if (left >= right)
        return 0;

    int mid = left + (right - left) / 2;

    int count = 0;

    count += mergeSort(nums, left, mid);
    count += mergeSort(nums, mid + 1, right);

    count += merge(nums, left, mid, right);

    return count;
}

int merge(int* nums, int left, int mid, int right) {
    int count = 0;

    // Count reverse pairs
    int j = mid + 1;

    for (int i = left; i <= mid; i++) {
        while (j <= right && (long long)nums[i] > 2LL * nums[j]) {
            j++;
        }

        count += (j - (mid + 1));
    }

    int size = right - left + 1;
    int* temp = (int*)malloc(size * sizeof(int));

    int i = left;
    j = mid + 1;
    int k = 0;
```

```

while (i <= mid && j <= right) {
    if (nums[i] <= nums[j]) {
        temp[k++] = nums[i++];
    } else {
        temp[k++] = nums[j++];
    }
}

while (i <= mid) {
    temp[k++] = nums[i++];
}

while (j <= right) {
    temp[k++] = nums[j++];
}

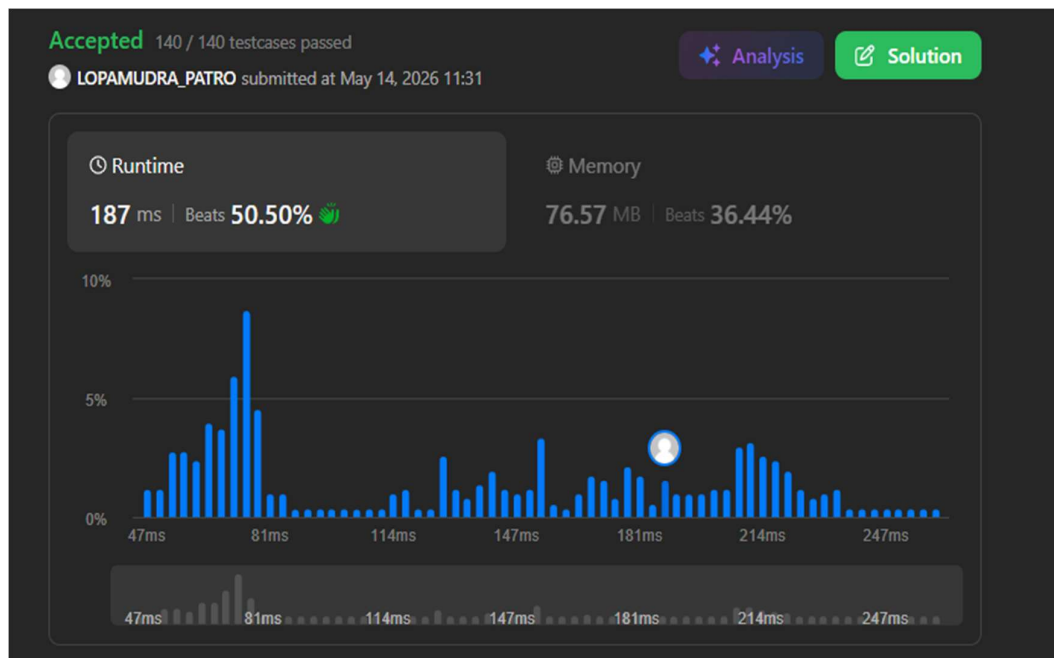
for (i = left, k = 0; i <= right; i++, k++) {
    nums[i] = temp[k];
}

free(temp);

return count;
}

```

OUTPUT



☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[1,3,2,3,1]

Output

2

Expected

2

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[2,4,3,5,1]

Output

3

Expected

3

Leetcode Problem 133. Clone Graph (Graphs)

```
struct Node* map[101];

struct Node* dfs(struct Node* node) {
    if (node == NULL)
        return NULL;

    if (map[node->val] != NULL)
        return map[node->val];

    struct Node* clone = (struct Node*)malloc(sizeof(struct Node));
    clone->val = node->val;
    clone->numNeighbors = node->numNeighbors;
    clone->neighbors = (struct Node**)malloc(
        clone->numNeighbors * sizeof(struct Node*)
    );

    map[node->val] = clone;

    for (int i = 0; i < node->numNeighbors; i++) {
        clone->neighbors[i] = dfs(node->neighbors[i]);
    }

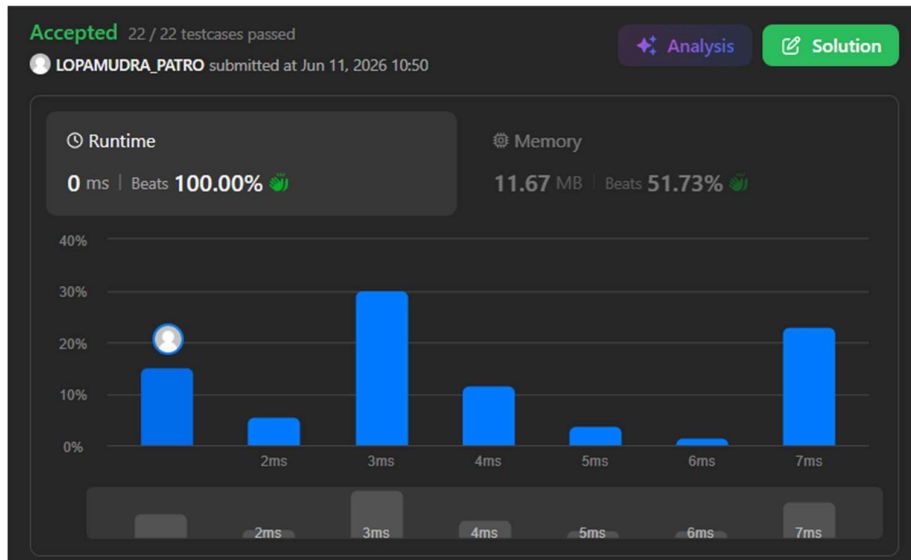
    return clone;
}

struct Node *cloneGraph(struct Node *s) {
    if (s == NULL)
        return NULL;

    for (int i = 0; i < 101; i++)
        map[i] = NULL;

    return dfs(s);
}
```

OUTPUT



☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
edges =  
[[2,4], [1,3], [2,4], [1,3]]
```

Output

```
[[2,4], [1,3], [2,4], [1,3]]
```

Expected

```
[[2,4], [1,3], [2,4], [1,3]]
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
edges =  
[[]]
```

Output

```
[[]]
```

Expected

```
[[]]
```

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

```
edges =  
[]
```

Output

```
[]
```

Expected

```
[]
```